Ingegneria del Software — Soluzione del Tema 05/07/2024

Es.1	Es.2	Es.3	Es.4	**Consegnare SOLO i fogli del tema.** *Non separarli. Altri fogli saranno ignorati.* I fogli sono fronte/retro. 2 ore di tempo. Tutti i dispositivi elettronici sono vietati.	**Attaccare l'etichetta in questa zona.** Compiti senza etichetta non sono validi. Si possono consultare libri e appunti.

Esercizio 1 Si consideri la seguente classe `UniqueList` che definisce una lista immutabile e senza elementi duplicati di oggetti confrontabili tra loro (interfaccia `Comparable`).

```
public /*@ pure @*/ class UniqueList {
  // Ritorna un Optional che contiene la testa della lista. Se la lista e' vuota, ritorna un Optional vuoto.
  // Si ricorda che un Optional e' vuoto se e solo se il suo metodo isPresent() ritorna false.
  public Optional<Comparable> head();

  // Ritorna la coda della lista, ovvero la sottolista di tutti gli elementi
  // ad esclusione della testa. Ritorna null in caso di lista vuota.
  public UniqueList tail();

  // Ritorna una nuova lista che ha l'elemento passato come parametro in testa e la lista originale in coda.
  // Lancia una DuplicateException se l'elemento e' gia' incluso nella lista.
  public UniqueList append(Comparable el) throws DuplicateException;

  // Ritorna vero se la lista e' vuota o se gli elementi presenti nella lista sono ordinati.
  // Si ricorda che l'interfaccia Comparable possiede un unico metodo compareTo(Comparable o)
  // che ritorna un intero negativo, zero o positivo a seconda che l'oggetto corrente sia minore,
  // uguale o maggiore dell'oggetto o passato come parametro.
  public boolean isOrdered();
}
```

Domanda a) Si specifichi in JML il metodo `append`.

Soluzione

NB: la classe UniqueList non implementa o eredita dalle Collections di Java: non ha quindi metodi come ad esempio `contains, get`.

Per scrivere la specifica è necessario quindi definire ad esempio un un predicato `contains` così specificato:

```
//@ contains(list, el) <=> ( list.head().isPresent() &&
//@   (list.head.get().equals(el) || contains(list.tail(),el) )
```

Usando tale predicato possiamo specificare, in modo molto semplice, il metodo `append` come segue:

```
//@ requires el != null
//@
//@ ensures !contains(this, el) &&
//@ \result.head().isPresent() && \result.head().get().equals(el) &&
//@ \result.tail().equals(this)
//@
//@ signals (DuplicateException e) contains(this, el)
public UniqueList append(Comparable el) throws DuplicateException;
```

NB: il metodo equals non è stato definito per la classe, quindi essa eredita la versione di Object, che è equivalente a "==". Una UniqueList è pura, quindi quindi la condivisione non crea problemi. Non è perciò errato ritornare l'oggetto this come tail.

Domanda b) Si specifichi in JML il metodo `isOrdered`.

Soluzione

È sufficiente una semplice ricorsione sulla tail:

```
//@ requires true
//@
//@ ensures \result == ( !head().isPresent() ||
//@   ( head().isPresent() && !tail().head().isPresent() ) ||
//@   ( head().get().compareTo(tail().head().get())<=0 && tail().isOrdered() ) )
public boolean isOrdered();
```

Domanda c) Si consideri l'implementazione di `UniqueList` che fa uso di una lista `elements` per salvare gli elementi:

```
public class UniqueList {
  private List<Comparable> elements;
  ... }
```

Per tale implementazione, si forniscano l'invariante di rappresentazione e la funzione di astrazione.

Soluzione

L'invariante di rappresentazione richiede che la lista non sia nulla, che non contenga elementi nulli o duplicati.

```
//@ private invariant
//@ elements != null && !elements.contains(null) &&
//@ (\forall int i; i>=0 && i<elements.size();
//@   (forall int j; j>=0 && j<i; !elements.get(i).equals(elements.get(j)) ) )
```

La funzione di astrazione deve definire i due metodi pubblici `head` e `tail` sulla base della lista `elements`. Tutti gli altri metodi pubblici sono infatti definiti a partire da questi due.

Per definire tali metodi, è necessario mettere in relazione gli elementi nelle stesse posizioni. Introduciamo quindi ancora un predicato ricorsivo `sameAs` che confronta una `List` e una `UniqueList` e ritorna `true` se e solo se contengono gli stessi elementi nello stesso ordine.

```
//@ sameAs(List l, UniqueList u) <==>
//@   (l.isEmpty() && !u.head().isPresent() && u.tail() == null) ||
//@   (!l.isEmpty() && u.head().isPresent() &&
//@     u.head().get().equals(l.get(l.size() - 1)) &&
//@     u.tail() != null && sameAs(l.subList(0, l.size() - 1), u.tail()) )
```

La funzione di astrazione viene quindi definita sulla base del predicato `sameAs`.

```
//@ private invariant sameAs(elements, this)
```

Domanda d) Si consideri una classe `RemovableUniqueList` (immutabile) che aggiunge un metodo `remove(Comparable el)` che ritorna una nuova lista in cui è stato rimosso l'elemento `el` (nel caso in cui questo fosse presente). Si può definire `RemovableUniqueList` come estensione di `UniqueList` secondo con il principio di sostituzione? Motivare la risposta.

Soluzione

`RemovableUniqueList` può essere definita come estensione di `UniqueList` secondo il principio di sostituzione, in quanto si tratta di un'estensione pura (aggiunta di un metodo) che non modifica le proprietà della classe `UniqueList`. La classe rimane infatti immutabile e il metodo genera una nuova lista invece di modificare (rimuovere un elemento) da quella su cui è stato invocato.

Esercizio 2

Si consideri la seguente classe `HydraulicSystem` che descrive un sistema idraulico chiuso, composto da un insieme di serbatoi d'acqua (`WaterTank`) connessi tra di loro.

Il metodo `swap` consente di spostare acqua da un serbatoio all'altro, mentre il metodo `totalWater` calcola il totale di acqua presente nel sistema.

```
public class HydraulicSystem {
  private List<WaterTank> tanks;

  public HydraulicSystem(int numTanks) {
    tanks = new ArrayList<WaterTank>();
    for(int i=0; i<numTanks; i++) {
      tanks.add(new WaterTank(1000));
    }
  }

  public void swap(int tank1, int tank2, int water) {
    int waterTaken;
    waterTaken = tanks.get(tank1).take(water);
    tanks.get(tank2).put(waterTaken);
  }

  public int totalWater() {
    int tot = 0;
    for(WaterTank wt : tanks) tot += wt.getWaterLevel();
    return tot;
  }
}

public class WaterTank {
  private int waterLevel;

  public WaterTank(int water) { waterLevel = water; }

  public synchronized void put(int water) {
    waterLevel += water;
  }

  public synchronized int take(int water) {
    if(water > waterLevel) water = waterLevel; // not enough water
    waterLevel -= water;
    return water;
  }

  public synchronized int getWaterLevel() { return waterLevel; }
}
```

Domanda a)

Nel suo insieme il codice evita il verificarsi di accessi concorrenti a zone condivise di memoria anche se i vari metodi fossero invocati da thread diversi?

Motivare la propria risposta e in caso di risposta negativa spiegare cosa andrebbe cambiato per evitare conflitti nell'accesso a memoria condivisa.

Soluzione

Il codice è correttamente sincronizzato per quanto rigaurda laa classe `WaterTank`, che ha tutti i metodi sincronizzati. In `HydraulicSystem` la cui lista `tanks` è immutabile. Si vede però anche il punto seguente.

Domanda b) Data una istanza `myHS` della classe `HydraulicSystem`, è corretto affermare che, anche in presenza di thread multipli, la chiamata `myHS.totalWater()` restituisce sempre il medesimo valore? Motivare la propria risposta e in caso negativo spiegare come cambiare il codice proposto, al fine di offrire tale garanzia.

Soluzione

Poichè il metodo `swap` non esegue atomicamente, è possibile che il codice eseguito a seguito della chiamata `myHS.totalWater()` si inframmezzi all'esecuzione, da parte di un altro thread, di `myHS.swap` ed in particolare a metà tra la fase di prelievo di acqua dal serbatoio sorgente e il versamento della medesima quantità di acqua nel serbatoio destinazione, restituendo un quantitativo di acqua totale diverso da quello reale.

Si può ovviare a tale comportamento rendendo sincronizzati i due metodi `swap` e `totalWater` della classe `HydraulicSystem`.

Domanda c) Data una istanza `myHS` della classe `HydraulicSystem`, si scriva il frammento di codice che crea ed avvia un thread che esegue all'infinito il seguente codice:

```
myHS.swap(1,2,10);
myHS.swap(2,1,10);
```

Soluzione

```
new Thread( () -> {
  while(true) {
    myHS.swap(1, 2, 10);
    myHS.swap(2, 1, 10);
  }
} ).start();
```

Esercizio 3 Si considerino le seguenti dichiarazioni di classi Java:

```
public class Paper {
  public String toString() { return "Paper"; }

  public void print(Paper a) {
    System.out.println("Paper: " + this + " " + compose(this,a));
  }

  public String compose(Paper a, Paper b) {
    return a.toString() + " and " + b.toString();
  }
}

public class Long extends Paper {
  public String toString(){ return "Long"; }

  public void print(Long a) {
    System.out.println("Long: " + this + " " + a);
  }
}

public class Short extends Paper {
  public String toString(){ return "Short"; }

  public void print(Short m) {
    System.out.println("Short: " + this + " " +  compose(this,m));
  }

  public String compose(Paper a, Paper b){
    return b.toString()+ " and " + a.toString();
  }
}
```

Domanda) Si consideri il seguente frammento di codice, che utilizza le classi sopra definite. Segnalare ed eliminare eventuali errori di compilazione o a run-time. Scrivere, per ogni riga che chiama il metodo `print`, il valore stampato durante l'esecuzione, *riportando sempre il numero di riga, pena annullamento*.

```
Paper p1, p2;
Short s1;
Long l1;
p1 = new Paper();
p2 = new Long();
s1 = new Short();
l1 = new Long();

1  s1.print(p1);
2  s1.print(p2);
3  s1.print(s1);
4  p2.print(p1);
5  p2.print(p2);
6  p2.print(s1);
7  l1.print(l1);
8  p2 = s1;
9  p1 = l1;
10 p2.print(p2);
11 p2.print(s1);
12 p1.print(l1);
```

Soluzione

Non vi sono istruzioni erronee. Il programma stampa:

```
1  Paper: Short Paper and Short
2  Paper: Short Long and Short
3  Short: Short Short and Short
4  Paper: Long Long and Paper
5  Paper: Long Long and Long
6  Paper: Long Long and Short
7  Long: Long Long

10 Paper: Short Short and Short
11 Paper: Short Short and Short
12 Paper: Long Long and Long
```

Esercizio 4 Si risolvano i seguenti esercizi usando esclusivamente i costrutti della programmazione funzionale in Java.

Domanda a)

Si completi il codice del seguente metodo che calcola la somma pesata di values con weights.

```
public static double calculateWeightedSum(List<Double> values, List<Double> weights) {
  if (values.size() != weights.size() || values.isEmpty()) {
    throw new IllegalArgumentException("Values and weights must have the same non-zero length.");
  }
  return ...
}
```

Un esempio di utilizzo del metodo è:

```
List<Double> values = List.of(1.0, 2.0, 3.0, 4.0, 5.0);
List<Double> weights = List.of(0.1, 0.2, 0.3, 0.4, 0.5);
double weightedSum = calculateWeightedSum(values, weights);
```

il valore di weightedSum in questo caso è: $0.1 + 0.4 + 0.9 + 1.6 + 2.5 = 5.5$.

Soluzione

```
return IntStream.range(0, values.size())
  .mapToDouble(i -> values.get(i) * weights.get(i))
  .sum();
```

Domanda b)

Si considerino le due seguenti classi `Person` e `Dish` (di cui sono riportate solo le parti rilevanti):

```
class Person {
  public Person(String name, List<Dish> preferredDishes) { .. }

  public List<Dish> getPreferredDishes() { .. }
}

class Dish {
  public Dish(String name, boolean vegan) { .. }

  public boolean isVegan() { .. }

  public String getName() { .. }
}
```

Completare il metodo `getVeganDishes`, dato di seguito, che restituisce, senza ripetizioni, la lista dei nomi dei piatti (dish) vegani preferiti dalla lista people passata come parametro.

Ad esempio, se la lista people è inizializzata come:

```
List.of(
  new Person("Alice", List.of(
    new Dish("Salad", true),
    new Dish("Pizza", false),
    new Dish("Burger", false))),
  new Person("Bob", List.of(
    new Dish("Pasta", false),
    new Dish("Salad", true),
    new Dish("Fries", true))),
  new Person("Charlie", List.of(
    new Dish("Burger", false),
    new Dish("Fries", true),
    new Dish("Apple Pie", true)))
);
```

il risultato della chiamata al metodo è la lista: `[Salad,Fries,Apple␣Pie]`

Il metodo è definito come:

```
public static List<String> getVeganDishes(List<Person> people) {
  return ...
}
```

Soluzione

```
public static List<String> getVeganDishes(List<Person> people) {
  return people.stream()
    .flatMap(person -> person.getPreferredDishes().stream())
    .filter(Dish::isVegan)
    .map(Dish::getName)
    .distinct()
    .collect(Collectors.toList());
}
```